

FIRMPASS: Identifying Broken Password Management in Linux-Based IoT Firmware Through Query-Driven Approaches

Jing Huang¹, Jiongyi Chen¹, Yuting Xiao¹, Zheng Qin¹, *Member, IEEE*,
and Yupeng Hu¹, *Senior Member, IEEE*

Abstract—Password management is a fundamental aspect of security for Internet of Things (IoT) devices. However, despite the availability of established guidelines and best practices, the implementation of password management in IoT firmware often falls short, leading to vulnerabilities and potential breaches. Because of the lack of automated tools, the severity and pervasiveness of broken password management of IoT firmware has been less understood. In this article, we present FIRMPASS, a new tool to identify broken password management of Linux-based IoT firmware. Particularly, we establish general password management models for Linux-based IoT devices based on password management processes and related vulnerabilities. To automatically identify the vulnerabilities, FIRMPASS employs a query-driven approach to locate the firmware that violate the properties of correct password management. Specifically, we manually define four rules that should be complied with, encode the rules with queries, and check the queries in the firmware, which leads to the discovery of four types of vulnerabilities. Evaluation of 615 IoT firmware images uncovers 67 vulnerabilities, including 37 previously unknown issues. Our work underscores the urgent need for assessment solutions for IoT firmware.

Index Terms—Authentication, password management, query-driven, vulnerability.

I. INTRODUCTION

PASSWORD management holds paramount significance in Internet of Things (IoT) device security, serving as the linchpin for authentication and authorization for user login. IoT devices rely on robust password management to safeguard against unauthorized access and cyber-attacks, ensuring data privacy and device security. Regrettably, there has been a notable surge in attacks caused by the broken password management in IoT devices [1], [2], [3], [4], [5], [6].

Received 18 January 2025; revised 7 March 2025 and 23 March 2025; accepted 11 April 2025. Date of publication 16 April 2025; date of current version 9 July 2025. This work was supported in part by the Yuelu Mountain Industrial Innovation Center Key Projects under Grant 2024YCH0110; in part by the National Key Research and Development Projects under Grant 2022YFB3103500; in part by the National Natural Science Foundation of China under Grant U22A2030, Grant 62441233, and Grant 62472166; in part by the Technology Projects of Hunan Province under Grant 2015TP1004; in part by the Natural Science Foundation of China under Grant 62302508; and in part by the Research Funding of NUDT under Grant ZK22-53. (Corresponding author: Yupeng Hu.)

Jing Huang, Yuting Xiao, Zheng Qin, and Yupeng Hu are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410012, China (e-mail: huangjjing@hnu.edu.cn; xyt990626@hnu.edu.cn; zqin@hnu.edu.cn; yphu@hnu.edu.cn).

Jiongyi Chen is with the National University of Defense Technology, Changsha 410073, China (e-mail: chenjiongyi@nudt.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3561520

Password management of IoT devices primarily includes password verification and storage during password-based authentication. Developers' insufficient understanding of password management during the development of device firmware can lead to security issues concerning privacy and security. Specifically, in the process of password verification, vulnerabilities such as plaintext storage of a password and weak encoding for password may arise if passwords are stored or transmitted without proper cryptographic protection, or if weak encoding schemes are used. These flaws could expose sensitive credentials to attackers. In the process of password storage, security issues like the use of hardcoded passwords can result from directly embedding fixed passwords or credentials into the firmware, leading to authentication bypass. Additionally, unverified password change allows attackers to change passwords without proper user authentication. The above issues are mainly caused by the lack of standards and awareness of best practices for the password management of IoT devices.

The automation of identifying password management issues in IoT devices is still in its infancy. There is currently a lack of comprehensive understanding regarding the security properties that should be adhered to. Moreover, developing program analysis techniques to accurately express these security properties as code properties for identifying broken password management remains a challenge. Existing research has made some strides in automatically revealing password management issues in IoT devices. This includes detecting authentication bypass vulnerabilities in firmware [7], identifying robust protections against authentication failures [8], [9], backdoor identification in binaries [10], performing security checks for Web user permission verification [11], and detecting misuse of cryptographic functions [12], [13]. However, these solutions often overlook the complete logical flow of password management and cannot be directly applied to the specific scenario. For example, *Firmalice* [7] centers on the authentication process and proposes a model for identifying authentication bypass. This approach lacks a comprehensive understanding of the entire logical flow involved in the authentication process within real-world firmware images. Consequently, the authentication bypass model is oversimplified and has only been validated on a limited number of firmware images.

In this article, we provide a comprehensive view of password management issues and develop FIRMPASS, an automatic firmware analysis tool designed to expose password

management vulnerabilities for Linux-based IoT firmware leveraging a query-driven approach. Specifically, we establish a password management model grounded in key security rules derived from basic password-based authentication, various CVEs, and vendor vulnerability reports. To implement the model, we formalize the basic facts and predicates of these password rules using Datalog and define a set of query languages tailored for the password management model of IoT devices. This enables precise representation of vulnerable code behaviors. The logical behaviors that conform to the query language defined by the model may potentially exhibit these vulnerabilities, allowing for effective identification.

We implemented FIRMPASS and evaluated it on 615 firmware from 14 vendors across multiple architecture types. The evaluation shows that FIRMPASS is effective in identifying broken password management for Linux-based IoT firmware. FIRMPASS successfully identified 67 vulnerabilities, including 37 previously unknown vulnerabilities. FIRMPASS identified 67 vulnerabilities in 615 devices with a minimum detection time of 11 s and a maximum detection time of 47 s. The accuracy of detection is 81.7%.

Contributions: The contributions of this article are summarized as follows.

- 1) *New Model:* We give a password management model and the security rules that should be followed for basic password-based authentication of Linux-based IoT firmware.
- 2) *New Approach:* On the basis of the password management model, we define queries to formalize the vulnerable code behaviors mapped from the security rules, and develop FIRMPASS to automatically detect four particular types of vulnerabilities for the password management issues of IoT firmware.
- 3) *Real-World Impact:* We developed a prototype of FIRMPASS on 615 firmware, which successfully identified 67 vulnerabilities, including 37 of them were previously unknown.

II. BACKGROUND

This section gives the necessary background on basic password-based authentication and password management process.

A. Basic Password-Based Authentication

Basic password-based authentication is a common approach to verify user identity and is widely used in Linux-based IoT firmware as shown in Fig. 1. It mainly includes two types: 1) unix password authentication for local login and 2) HTTP authentication for remote Web users.

In Linux-based IoT firmware, the Unix password authentication is used for daemons or services like *busybox*, *telnet*, and *ssh*, which typically leverages the pluggable authentication modules (PAM) framework [14]. The frontend service invokes the PAM framework to authenticate users by comparing the provided credentials with the values stored in the */etc/passwd* and */etc/shadow* files. In a Unix-like system, the */etc/passwd*

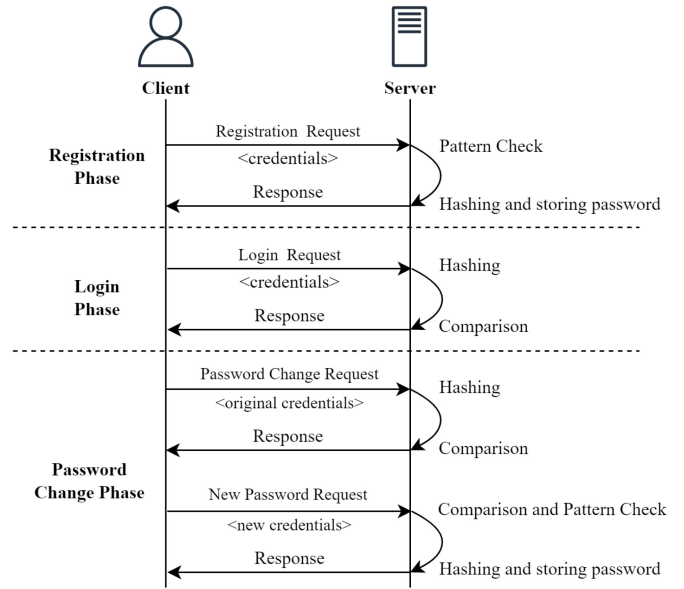


Fig. 1. Basic password-based authentication in linux-based iot firmware.

file contains user account information, while the */etc/shadow* file contains the hashed values of user passwords.

For Web servers in IoT firmware, the HTTP offers a versatile framework for managing access privileges and authentication mechanisms. The HTTP authentication includes the basic authentication, the digest authentication, and the form-based authentication. Each with its own balance of security strength and ease of implementation in client or server software. As an example, for Apache, a common approach to password protection involves the use of *.htaccess* and *.htpasswd* files. These files serve a similar purpose to the Unix */etc/passwd* and */etc/shadow* files but are tailored for Web environments. The *.htaccess* file directs to a *.htpasswd* file, which contains username-password pairs, each separated by a colon. The passwords are not displayed in plaintext; instead, they are stored as hashes, typically using SHA-256 and SHA-512. Additionally, for applications like *httpd* in IoT firmware, the credentials that are provided at the stage of user registration are sometimes stored in nonvolatile random-access memory (NVRAM), rather than in system files.

B. Password Management Process

Fig. 1 illustrates the password management process of basic password-based authentication. The password management process co-exists with the authentication process, which can be divided into three main phases: 1) registration phase; 2) login phase; and 3) password change phase.

Registration Phase: The server filters out incorrect password forms by pattern check and records the username and password of the user in this phase. In the process of pattern check, robust rules are set to handle the length of the password, the complexity of the password format, password retrieval issues, and the comparison of historical passwords. Cryptographic algorithms are used for the credentials' transmission and storage on the server. After the user sets a new password, the server updates the stored credential.

```

1 undefined4 UndefinedFunction_00011328(undefined4 param_1,
  int param_2){
2   ppcVar7 = (char **)FUN_0000f6e8(pcVar13);
3   puVar9 = (undefined1 *)FUN_00059fec(0, "Old password:");
4   pcVar10 = __s1_00;
5   __s2 = __s1_00;
6   if (puVar9 != (undefined1 *)0x0) {
7     __s1 = (char *)FUN_00061bc4(puVar9, ppcVar7[1], 1);
8     iVar8 = strcmp(__s1, ppcVar7[1]);
9     if (iVar8 == 0) goto LAB_00011540;
10    syslog(4, "incorrect password for %s", *ppcVar7);
11    FUN_0005a188(3);
12    puts("Incorrect password");}}

```

Listing 1. Code snippet from the device model PLANEX CS-W50HD.

Login Phase: In this phase, user credentials are delivered to the server and compared with the information from the registration phase. Cryptographic algorithms such as hashing are again involved. For local login, successful user authentication would allow users to privileged operations. For remote login, an authentication token is returned to the client for requesting resources.

Password Change Phase: When users change their password, they are involved in the password change phase. Before entering a new password, users must be authenticated by providing their old password. The new password undergoes the same pattern checks as in the registration phase. After that, the new password is hashed and compared with the user's original hashed password. Finally, once the new password is different from the old one, the server updates the storage with the hash of the new password.

III. MOTIVATION

In this section, we give the scope of this research, use a practical example to illustrate the problem, and introduce the threat model.

A. Problem Scope

The focus of this research is identifying broken password management in Linux-based IoT firmware. The password management process includes the input, the encryption, the storage, and the update of passwords. Particularly, we focus on password management for basic password-based authentication, which is widely adopted by typically services such as *httpd*, *ftpd*, *sshd*, and *busybox* under the Linux platform.

B. Running Example

To better understand the problem, we use the code snippet of the program *busybox* of the device model PLANEX CS-W50HD as a running example. Listing 1 shows the authentication procedure required for changing a user's password. The stored hash user password *ppcVar7* is obtained through the username *pcVar13* in line 2. The program extracts the plaintext password input by the user and stores it into *puVar9* in line 3. In lines 7 and 8, the plaintext password *puVar9* is hashed and compared to the stored hashed password *ppcVar7*. The user can proceed with the follow-up password change if the comparison is consistent.

The procedure in Listing 1 reveals potential password management issues. Suppose the hash operation in line 7 uses

weak encoding or risky encryption algorithms, meaning the stored passwords are insecurely hashed. In that case, the hash passwords stored in the configuration file can be easily cracked by attackers.

C. Threat Model

Exploiting password management vulnerabilities can lead to broken access control, allowing attackers to access and steal sensitive data. Then, they can use the obtained sensitive information to elevate privileges. Attackers also can modify or corrupt system configurations, resulting in service disruption or data loss.

Threat Model for Plaintext and Weak Encoding: Systems that store or transmit credentials in plaintext or with weak encryption mechanisms are vulnerable to exploitation by attackers who seek to steal sensitive data. In this scenario, attackers can exploit client-side storage (e.g., configuration files, databases, or logs) or intercept network traffic to retrieve credentials directly. For instance, credentials stored without encryption (plaintext) or encoded using reversible methods like base64 are trivial for attackers to decode. Similarly, weakly encrypted credentials—such as those protected by outdated algorithms (e.g., DES), hard-coded encryption keys, or improper key management—can be decrypted using brute-force attacks, cryptanalysis, or by exploiting implementation flaws. Attackers may also sniff unencrypted network transmissions (e.g., over HTTP or misconfigured TLS) to harvest credentials in transit. Once obtained, these credentials enable attackers to impersonate legitimate users or systems, escalate privileges to access restricted resources, and manipulate or exfiltrate sensitive data. By leveraging stolen credentials, attackers can further modify system configurations, disrupt services, or delete critical data, leading to operational downtime and reputational harm. This threat model assumes attackers have direct access to client-side storage or network channels but does not require advanced capabilities beyond basic reverse-engineering or traffic interception tools.

Threat Model for Hard-Coded Password and Unverified Password Change: Systems that rely on hard-coded credentials or insecure password-change workflows are susceptible to authentication bypass and unauthorized access. Attackers in this model target credentials embedded directly in source code, firmware, or configuration files, which can be extracted through reverse-engineering binaries or inspecting client-side files. Hard-coded credentials, such as default administrative passwords, allow attackers to bypass authentication entirely and gain privileged access to both client and server interfaces. On the other side, unverified password change enables attackers to hijack accounts. For example, an attacker might reset passwords for arbitrary accounts. Once authenticated, attackers can escalate privileges, access sensitive data, and execute high-risk operations, such as modifying system configurations, deploying backdoors, or deleting data. This threat model assumes attackers can reverse-engineer software components or intercept insecure communication channels but does not presuppose prior access to privileged accounts. The persistence of hard-coded credentials further allows attackers to maintain

long-term control over compromised systems, leading to sustained data breaches or service disruptions.

IV. MANUAL ANALYSIS

This section introduces the common password management vulnerabilities in Linux-based IoT firmware, the rules that we define by analyzing the logical flow of correct password management, and the password management model that we derive.

A. Vulnerabilities

Password management involves the creation, secure storage of passwords, and the enforcement of password policies. For basic password-based authentication in Linux-based IoT devices, password management primarily includes the use of robust cryptographic algorithms for authentication and secure storage, and the update of passwords in hashed formats. Security risks could appear in the entire flow of password management. The issues are particularly outstanding for Linux-based IoT firmware. Upon login, the password provided by the user is usually hashed and compared against the stored credentials. In this step, if the credentials are stored in plaintext, this could pose a security risk, namely CWE-256: Plaintext Storage of a Password. If the chosen hashing algorithms to encode or encrypt the password before storage are faulty, CWE-261: Weak Encoding for Password is introduced. Notably, the stored password, either in plaintext or in hash format, could be hardcoded in the program context. This results in CWE-259: Use of Hard-coded Password. Changing or updating the password is also a significant part of password management. Before changing a password, the device usually verifies the users' identities, typically by validating their usernames and passwords. A risk is to change the password without verifying the user's identity first, namely CWE-620: Unverified Password Change.

The details of the four types of vulnerabilities that we aim to address are given as follows.

- 1) *Type 1 (CWE-256, Called Plaintext Storage of a Password)*: It refers to the insecure practice of storing passwords in an unencrypted form, either in configuration files or memory, which makes them vulnerable to unauthorized access.
- 2) *Type 2 (CWE-261, Called Weak Encoding for Password)*: It refers to the inadequate practice of obscuring passwords using trivial encoding methods, such as base64, which can easily be reversed and does not provide sufficient security against unauthorized access.
- 3) *Type 3 (CWE-259, called Use of Hard-Coded Password)*: It refers to the insecure practice of embedding fixed credentials directly into a device or software, which are used for authentication or communication, making it difficult to detect and fix, and potentially compromising system security.
- 4) *Type 4 (CWE-620, Called Unverified Password Change)*: It refers to the insecure practice of allowing a password

to be changed without properly verifying the user's identity, which can lead to unauthorized access.

B. Security Rules

The security rules for password management in Linux-based IoT devices are derived by analyzing common vulnerabilities and their weaknesses, as highlighted by the industry-standard CWE categories. By manually examining these vulnerabilities, we can establish security practices that can mitigate these risks.

Rule 1: When the user logs in, the user's input needs to be verified by comparing it with the hashed password. This security rule addresses the need for proper handling and verification of user credentials during the login process. Storing passwords in plaintext poses a serious security risk. Hence, passwords should always be hashed before storage, and during the login process, the plaintext password input by the user should always be hashed before compared against the stored password.

The vulnerability CWE-256 points out the danger of storing passwords in plaintext. Devices should ensure that user credentials are hashed securely. Therefore, this rule enforces the correct procedure for password comparison: the user's input should not be compared directly with plaintext but with stored password securely hashed.

Rule 2: The hash algorithm used to hash user input during user login authentication must be highly secure. This rule emphasizes the need for using strong cryptographic hashing algorithms in the crypto library when hashing the user's input during the authentication process. Weak or easily reversible encoding methods, such as base64 and md5, are insufficient and introduce CWE-261. Secure hash algorithms, such as SHA-256, should be employed to ensure that the stored password hashes cannot be easily cracked.

Rule 3: During the password authentication process, the stored password can be updated and changed. This rule ensures that passwords used for authentication are dynamically managed rather than hard-coded in the firmware. The use of hard-coded passwords introduces vulnerabilities (namely CWE-259) that are difficult to patch and compromise the overall security of the device. Therefore, passwords should be allowed to be securely updated and changed as needed.

Rule 4: Users must be authenticated with the old password to gain access to change the new password. This rule aims to prevent unauthorized password changes without verifying users' identities. CWE-620 highlights the risk of such changes, leading to compromising the device. To mitigate this, a system must authenticate users with their existing password (or another form of strong authentication) before allowing a password change.

C. General Model

The password management model for Linux-based IoT devices can be described as a structured framework that connects high-level security rules to low-level code semantics in Fig. 2. This model governs the handling of user credentials during the registration phase, login phase, and password change phase, following the security rules derived from

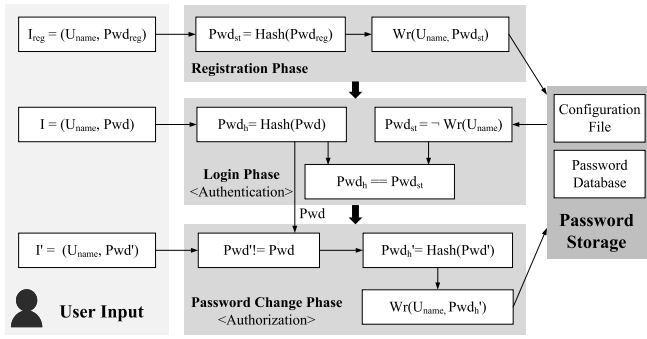


Fig. 2. General model of password management.

common vulnerabilities in password management. The model ensures that cryptographic operations, password updates, and user authentication are securely executed, maintaining a closed-loop process throughout the lifecycle of password management.

Registration Phase: During the registration phase, the system ensures that user input I_{reg} meets security requirements. This phase begins with applying pattern checks on the password Pwd_{reg} to filter out weak passwords based on their length, complexity, and uniqueness. These checks are crucial for enforcing robust password policies, aligning with Rule 2 in Section IV-B, which requires using the secure hashing algorithm, and Rule 4, ensuring users cannot register with weak or recycled passwords.

Once a password passes these checks, it is hashed using a secure cryptographic algorithms $Hash(Pwd_{reg})$. The hashed password Pwd_{st} is then securely transmitted and stored $Wr(U_{name}, Pwd_{st})$ by the system, adhering to Rule 1, which requires the user's password to be stored in hashed form. The strength of the cryptographic algorithm is essential. The hashed credentials are stored in preparation for future authentication.

Login Phase: In the login phase, the system authenticates users by verifying their credentials. When a user enters its identity $I(U_{name}, Pwd)$, the password input is hashed using the same algorithm used during registration. The hashed input Pwd_h is compared against the stored hashed password Pwd_{st} , which is obtained by a fetch operation $\neg Wr(U_{name})$.

This process aligns with Rule 1, which ensures user credentials are compared against the securely hashed passwords stored in configurations or databases. The cryptographic algorithm used in login should be highly secure to prevent risks outlined in CWE-261, following Rule 2. For local login, successful authentication grants users access to privileged operations, while in remote login, an authentication token is returned to facilitate secure resource requests.

Password Change Phase: The password change phase focuses on securely updating the user's credentials $I(U_{name}, Pwd')$. Before the password change, the user should pass the login phase first, ensuring that only authorized users U_{name} can modify their credentials. This authentication step complies with Rule 4, which requires users to be verified with the old password before allowing changes.

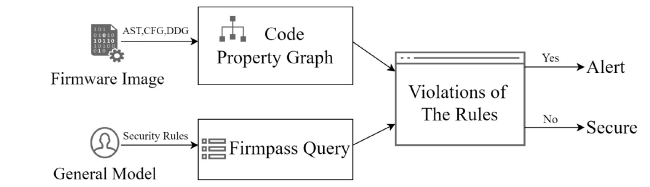


Fig. 3. Overview of FIRMPASS.

Once authenticated, the new password Pwd' undergoes a pattern check similar to the registration phase to ensure it meets security standards. The new password is compared to the old password Pwd to confirm it is different, which avoids reusing old credentials. If the new password passes these checks, the system updates the stored password hash Pwd'_h , completing the change process. This phase adheres to Rule 3, ensuring that password updates are conducted securely and Rule 2, ensuring that strong cryptographic methods are applied.

In summary, this password management model bridges high-level rules with low-level code operations by defining the specific steps and logical flows required to handle passwords securely. For example, the model ensures that password hashing, comparison, and storage operations are aligned to corresponding security rules (such as verifying passwords with hashed versions). It captures the entire logical flow from user input, through cryptographic operations, to secure storage, ensuring each phase in the password lifecycle adheres to best practices, including secure hashing, authentication, and update processes.

V. DESIGN OF FIRMPASS

In this section, we give the design details of FIRMPASS, of which the overview is shown in Fig. 3. FIRMPASS analyzes IoT firmware images by extracting the binaries and converting them into code property graphs (CPGs). These graphs capture both the structure and semantics of the binary code. FIRMPASS defines custom query languages that represent security rules, allowing it to verify the properties of the extracted code effectively. By doing so, it systematically identifies the vulnerabilities caused by broken password management.

A. Code Representation

We formalize logical processes of authentication and authorization in password management by using the query language to define the code property characteristics of password management rules.

Code Property Graph: FIRMPASS utilizes the CPG to represent data flows and control flows of the program. We use Joern [15] to construct the intermediate structure of CPG as the processing target, which is a kind of graph for joint representation that combines abstract syntax tree (AST), control flow graph (CFG), control dependence graphs (CDG), data dependence graphs (DDG), and program dependence graph (PDG).

Facts and Inference Rules for CPG: Inspired by the formalization of Cerberus [16], FIRMPASS converts the CPGs of the

program into its corresponding facts and inference rules using Datalog. We represent the code properties, function contents, branch conditions, and calling relationships by datalog, which are included in the security rules. We first give some preliminaries on the Datalog program and present the syntax and semantics of FIRMPASS's built-in facts.

A Datalog program consists of facts and rules written in a declarative logic language. For example, in the context of CFG's inference rules, $\text{path}(\text{stmt1}, \text{stmt2})$ implies that statement node stmt1 and stmt2 reside within a unified dataflow path where stmt1 and stmt2 are reachable via invocation-chain dependencies through function call. Each rule is a Horn clause [17] defining a predicate as a conjunction of predicates. In addition to variables, predicates can also contain constants. For example, the following program is represented by Datalog:

$$\begin{aligned} \text{depend}(x_1, x_2) &:- \text{call}(x_1, x_2) \\ \text{depend}(x_1, x_2) &:- \text{depend}(x_1, x_3), \text{call}(x_3, x_2) \end{aligned}$$

where $\text{depend}(x_1, x_2)$ is true if $\text{call}(x_1, x_2)$ is true, or both $\text{depend}(x_1, x_3)$ and $\text{call}(x_3, x_2)$ are true. Facts correspond to predicates that are evaluated as true.

Base Facts in FIRMPASS: According to CPG, many crucial properties in the Datalog require a flow-sensitive analysis to identify whether the execution logic on the critical flow is reasonable.

Therefore, FIRMPASS's facts take the form of $N(I, L, Y, x_1, \dots, x_n)$, where N is the instruction name, I is the instruction id, L is the instruction label, Y is the variable storing the instruction result (if any), and $x_1, \dots, x_n$ are variables passed to the instruction as arguments (if any). For instance, the instruction $x = 0$ is encoded as $\text{assign}(256, \text{Call}, x, 0)$.¹ The branch instruction $\text{branch}(P_1, X, P_2, P_3)$ denotes that if X is evaluated to true, then the next path for execution will be P_2 , otherwise P_3 . Using the base facts described above, FIRMPASS computes the semantic facts of: 1) *CFG-Related*: control-dependency predicates and 2) *DDG-Related*: data-dependency predicates.

CPG Predicates: FIRMPASS provides built-in predicates to encode CPG generated from code representation, especially the encoding of flow-insensitive predicates. According to DDG, the predicate $\text{dependOn}(x, y)$ indicates that the value of variable y has a data dependency on x . Similarly, the predicate $\text{followBy}()$ is inferred from the CFG of programs. $\text{FollowBy}(x, y)$ means that path y follows path x or statement x is called by program statement y . Predicate $\text{compara}(x, y)$ represents a comparison between x and y . All of the above predicates are flow-sensitive, such as

$$\text{followBy}(x, y) :- \text{followBy}(z, y), \text{followBy}(x, z).$$

FIRMPASS Predicates: The query language module of FIRMPASS also defines a list of predicates specific to password management in Table I. These predicates represent the dependency relationship between data flow and control flow, as well as the unique logical

¹For simplicity, this article omits the id, label and represents $\text{assign}(256, \text{Call}, x, 0)$ as $\text{assign}(x, 0)$.

TABLE I
TYPE OF PASSWORD MANAGEMENT PREDICATES

Predicates	Description
$\text{isUserCount}(x)$	X may be user's input in busybox
$\text{isWeak}(x)$	Weak hash algorithm x
$\text{cryptStruct}(x)$	Cryptographic library algorithm x
$\text{encode}(x)$	Encode x by base64
$\text{isToken}(x)$	X maybe users input in remote login
$\text{isLiteral}(x)$	X may be literal
$\text{isSecret}(x)$	X maybe stroe passwd in remote login
$\text{hash}(x)$	Hash input x
$\text{passwdStruct}(x)$	X points to "passwd" files
$\text{obsucre}(x)$	Check the format of password x
$\text{shadowStruct}(x)$	X points to "shadow" files
$\text{configStruct}(x)$	X points to configure files
$\text{update}(x)$	Update the stored password files
$\text{error}(x)$	An error message x
$\text{isStoreCredentials}(x)$	X maybe credentials in local login

process in password management. For example, we use $\text{isUserCount}(x)$, $\text{isStoreCredentials}(x)$, $\text{isToken}(x)$ and $\text{isSecret}(x)$ to identify the user input and storage operations of the programs. In the password verification, the predicate that identifies the type of the password file is $\text{passwdStruct}(T)$ and $\text{shadowStruct}(T)$. The predicate for identifying the hash of a user's password has $\text{hash}(x)$, $\text{cryptStruct}(x, y)$, and $\text{isWeak}(x)$.

B. Query Language

FIRMPASS maps the high-level rules in Section IV-B to queries. It expresses password management properties over the semantics facts of the program and combines them into a complete query statement. After we derive the specific queries, we give an analysis on how the unsatisfaction of the specific queries could lead to the discovery of different types of vulnerabilities.

We first define the query language for expressing the security rules. The syntax of the query language is given by the following Backus-Naur form [18]

$$\text{symbol} ::= _expression_.$$

This construction enables the identification of whether a data flow adheres to a specified security rule. *Symbol* represents a security rule, and *expression* represents a data flow of the security rule. Specifically, for each security rule, we define a set of unique predicate combinations to represent the proper password management process. We also summarize the generic predicate combinations for the four types of vulnerabilities as well and use the query language to express the possible vulnerability paths, indicating the presence of vulnerabilities in the data flow, as shown in Table II.

Next, we briefly present the violation of rules and their formal representations as outlined in Tables I and II.

Violation of Rule 1: We first identify two passwords input and stored shown in Fig. 2 as sources by querying the predicates. Then, we trace all the paths of $\text{compara}(x, y)$ in CPG. If the intersection of the two sources in the $\text{compara}(x, y)$ is identified, it indicates a comparison. If the $\text{hash}(x)$ query is absent, it means the user input and stored

TABLE II
DEFINITIONS OF THE RULE VIOLATION FOR THE FIRMPASS QUERY LANGUAGE

Rule	Description of the Rule	Vulnerability Type	FIRMPASS Query for Vulnerability
R1	During the authentication, the user's input must be hashed.	Plaintext Storage of a Password	$\text{isUserCount}(x_1) \wedge \text{isStoreCredentials}(y_1) \wedge \text{depenOn}(y_1, y_2) \wedge \text{passwdStruct}(y_2) \wedge \text{followBy}(y_2, y_3) \wedge \text{shadowStruct}(y_3) \wedge \text{compara}(x_1, y_3)$
R1	During the authentication, the user's input must be hashed by the hash algorithm.	Plaintext Storage of a Password	$\text{isUserCount}(x_1) \wedge \text{followBy}(x_1, x_2) \wedge \text{depenOn}(x_1, x_2) \wedge \text{encode}(x_2)$
R1	During the authentication, Passwords must be stored in a password file, not a configuration file.	Plaintext Storage of a Password	$\text{isUserCount}(x_1) \wedge \text{isStoreCredentials}(y_1) \wedge \text{depenOn}(y_2, y_1) \wedge \text{configStruct}(y_2) \wedge \text{compara}(x_1, y_1)$
R2	The hash algorithm must be a highly secure algorithm.	Weak Encoding for Password	$\text{hash}(x_2) \wedge \text{followBy}(x_2, x_3) \wedge \text{depenOn}(x_2, x_3) \wedge \text{cryptStruct}(x_3) \wedge \text{isWeak}(x_3)$
R3	During the authentication process, the type of credentials must be variable.	Use of Hard-coded Password	$\text{isUserCount}(x_1) \wedge \text{followBy}(x_1, x_2) \wedge \text{depenOn}(x_1, x_2) \wedge \text{hash}(x_2) \wedge \text{isStoreCredentials}(y_1) \wedge \text{branch}(L_y 1, \text{if}, L_y 2, L_e) \wedge \text{isLiteral}(\text{if}(\text{argu}))$
R3	During the authentication process, passwords must be stored in a password file and cannot be hard-coded in another file.	Use of Hard-coded Password	$\text{isUserCount}(x_1) \wedge \text{followBy}(x_1, x_2) \wedge \text{depenOn}(x_1, x_2) \wedge \text{hash}(x_2) \wedge \text{isStoreCredentials}(y_1) \wedge \text{compara}(x_2, y_1)$
R4	The old password must be authenticated before the new password can be authorized.	Unverified Password Change	$\text{isStoreCredentials}(y_1) \wedge \text{followBy}(y_1, y_2) \wedge \text{passwdStruct}(y_2) \wedge \text{followBy}(y_2, y_3) \wedge \text{shadowStruct}(y_3) \wedge \text{isUserCount}(n_1) \wedge \text{compara}(n_1, y_3) \wedge \text{isUserCount}(n_2) \wedge \text{compara}(n_1, n_2) \wedge \text{hash}(n_2) \wedge \text{update}(n_2)$

passwords compared by $\text{compara}(x, y)$ are plaintexts, indicating a Plaintext Storage of a Password vulnerability. It is formalized as follows:

$$\text{isUserCount}(x_1) \wedge \text{isStoreCredentials}(y_1) \wedge \text{depenOn}(y_1, y_2) \wedge \text{passwdStruct}(y_2) \wedge \text{followBy}(y_2, y_3) \wedge \text{shadowStruct}(y_3) \wedge \text{compara}(x_1, y_3)$$

Violations of Security Rule 1 that lead to plaintext vulnerabilities are of two kinds, in addition to the lack of a hash algorithm. One is encryption using base64 encoding and the other is storing user passwords in unencryption-protected configuration files, and their formal representations are in Table II.

Violation of Rule 2: We identify if a hash function is involved in the flow of the user password and if the hash is applied in the query statement. Usually, hash algorithms are stored in shared object libraries. To query hash algorithms across programs, we developed a query script. The script checks for the presence of selected weak hashing algorithms in the basic hash operation $\text{cryptStruct}(x)$ by using the query language to determine if a Weak Encoding for Password vulnerability exists. It is formalized as follows:

$$\text{hash}(x_2) \wedge \text{followBy}(x_2, x_3) \wedge \text{depenOn}(x_2, x_3) \wedge \text{cryptStruct}(x_3) \wedge \text{isWeak}(x_3)$$

Violation of Rule 3: The Use of Hard-coded Password vulnerability involves storing passwords in plaintext that cannot be changed, such as storing passwords in the program context. To detect this type of vulnerability, we use the query language in a three-step process.

Step 1: Locating the user password through the query language and determining whether it is a basic password-based authentication. we identify the authentication process λ according to the query language:

$$\text{isUserCount}(x_1) \wedge \text{followBy}(x_1, x_2) \wedge \text{depenOn}(x_1, x_2) \wedge \text{hash}(x_2) \wedge \text{compara}(x_2, y)$$

Step 2: Checking whether the query statement for basic password-based authentication includes hash algorithm

$\text{hash}()$ and verifying if the password storage file type is either $\text{passwdStruct}()$ or $\text{isSecret}()$. We should determine whether the password storage file $\text{shadowStruct}()$ and the update file $\text{update}()$ refer to the same file for basic password-based authentication.

Step 3: Outputting the path of stored passwords in the AST graph in JSON format. We focus on the $\text{branch}(P_1, X, P_2, P_3)$ query in the conditional judgment X. After filtering out the X in the comparison function by using the query predicate $\text{isLiteral}(X)$, we can determine if there is hard-coded in the parameters of the conditional judgment.

Violation of Rule 4: The proper procedure for changing a password is to require user authentication before granting authorization. Similar to violation of Rule 3, the query language can locate the login phase λ of basic password-based authentication. The login phase λ runs through the query language $\text{branch}(L_\lambda, \text{if}, L_\delta, L_e)$ to the password change phase δ , the process of δ as follows:

$$\text{isUserCount}(n_1) \wedge \text{compara}(n_1, y) \wedge \text{isUserCount}(n_2) \wedge \text{compara}(n_1, n_2) \wedge \text{hash}(n_2) \wedge \text{update}(n_2)$$

A complete query path of the login phase to the change password phase confirms the absence of Unverified Password Change vulnerabilities within the logical process. Conversely, the lack of the authentication process indicates a security risk.

VI. IMPLEMENTATIONS AND EVALUATION

In this section, we give the implementation details of FIRMPASS and the evaluation results based on real-world IoT firmware.

A. Implementation and Setup

Implementation: We have implemented FIRMPASS with around 1793 Java lines of code in total based on logic programming [19]. Also, several open-source projects like Binwalk [20], Ghidra [21], and Joern [15] are integrated into the prototype implementation of FIRMPASS. Specifically, FIRMPASS preprocesses the firmware with Binwalk [20] to obtain the binaries. Ghidra [21] is used to recover variables and

TABLE III
CONFIRMED VULNERABILITIES WITH CVE

Device ID	CVE	Identified Vulnerabilities
DIR-601	CVE-2018-10641	Plaintext Password Storage
DIR-640L	CVE-2018-10824	Plaintext Password Storage
WNR4500-V1	CVE-2016-5638	Plaintext Password Storage
DSL-2888A	CVE-2020-24577	Plaintext Password Storage
WRT54G-v4	CVE-2008-1263	Plaintext Password Storage
X9Series_AVer	CVE-2016-6537	Plaintext Password Storage
WRT1900ACS	CVE-2019-7311	Plaintext Password Storage
WRT1900ACS	CVE-2019-7311	Plaintext Password Storage
DIR-825	CVE-2018-18767	Plaintext Password Storage
US_AC6V1	CVE-2020-28093	Use of Hard-coded Password
RV110W	CVE-2020-3330	Use of Hard-coded Password
R6700-v3	CVE-2021-45077	Use of Hard-coded Password
DIR-850L	CVE-2017-14418	Use of Hard-coded Password
MOFI4500	CVE-2020-15833	Use of Hard-coded Password
CS-W50HD	CVE-2017-12574	Use of Hard-coded Password
SRS60	CVE-2020-11549	Use of Hard-coded Password
RAX40-v1	CVE-2021-20171	Use of Hard-coded Password
TL-WR840N	CVE-2019-6972	Weak Encoding for Password
RV32X	CVE-2019-1828	Weak Encoding for Password
FUAA	CVE-2019-9861	Weak Encoding for Password
EDR-810	CVE-2017-12129	Weak Encoding for Password
WRT54G	CVE-2005-2915	Weak Encoding for Password
DIR-825	CVE-2018-18767	Weak Encoding for Password
DIR130	CVE-2017-3192	Weak Encoding for Password
DIR-100	CVE-2013-7055	Unverified Password Change
DIR-601	CVE-2018-10641	Unverified Password Change
DIR-878	CVE-2019-9124	Unverified Password Change
DIR-100	CVE-2013-7052	Unverified Password Change
DIR-825	CVE-2019-9123	Unverified Password Change
DIR-867	CVE-2020-8864	Unverified Password Change
TL-WR1043ND	CVE-2019-6972	Inadequate Encryption Strength
InHand IR615	CVE-2021-38474	Inadequate Encryption Strength

TABLE IV
PART OF EVALUATED DEVICES

Vendors	Models	Architecture	Firmware Versions
D-Link	DIR-100	MIPS32	1-14_eu_en_20140124
D-Link	DIR-601	MIPS32	1.04
D-Link	DIR-640L	MIPS32	1.02B02
D-Link	DIR-825	MIPS32	2.05
D-Link	DIR-842	MIPS32	v3.00B18
D-Link	DIR-878	MIPS32	1.12B01
D-Link	DWR-711	MIPS32	A1_FW1.11
D-Link	DWR-921	MIPS32	C3V0101
D-Link	DSL-520B	MIPS32	1.10_NA
D-Link	DSL-2740B	MIPS32	2.06
D-Link	DSL-2888A	MIPS32	AU_2.29_V1.1.47ae48
Cisco	RV32X	MIPS64	v1.3.2.02_20160923
Cisco	RV110W	MIPS32	1.2.2.8
TP-Link	WR1043ND	MIPS32	V6.20_180614
TP-Link	WR840N	MIPS32	V6.20_180614
Linksys	WRT1900ACS	ARM EABI5	1.0.3.187766_prod
Linksys	WRT54G	MIPS32	4.21.5.000_20120220
Netgear	R6700	ARM EABI5	V1.0.4.120_10.0.91
Netgear	R8500	ARM EABI5	V1.0.2.86_1.0.75
Netgear	SRS60	ARM EABI5	V2.5.1.106
Netgear	RAX40	MIPS32	V1.0.3.88_1.0.1
Netgear	WNR4500	MIPS32	V1.0.1.40_1.0.68
Netgear	WAC124	MIPS32	V1.0.4.6
Planex	CS-W50HD	ARM EABI5	ver 030608
AVer	EH6108H	ARM EABI4	X9.03.24.00.071
ASUS	BRT-AC828	ARM EABI5	3.0.0.4
Tenda	AC15	ARM EABI5	V1.0BR
Tenda	AC6	ARM EABI5	V2.0RTL
Mofi	MOFI4500	MIPS32	4GXeLTE 4.1.5
TP-Link	WR1043ND	MIPS32	V2
InHand	IR615	MIPS32	2.3.0.r4724

functions of programs from binaries of different architectures and output decompiled results in the C-Code level for query-driven analysis. Moreover, we rely on Joern [15] to generate CPGs to represent code for cross-language code analysis. We transform the decompiled results output by Ghidra into a data flow consisting of facts and predicates, then match it against defined rules in query language to identify the broken password management.

Firmware Dataset: To verify the effectiveness and scalability of FIRMPASS, we collected 615 firmware from 14 well-known router vendors, such as D-Link, TP-Link, Netgear, Cisco, Linksys, Tenda, ASUS, 360, TOTOLINK, Buffalo, Mikrotik, Planex, AVer, and Mofi. We also collected different versions of the same firmware to rule out the impacts of specific versions. All evaluated targets are IoT firmware images downloaded from the official website. These firmware images include different architectures such as ARM (EABI4, EABI5), MIPS, MIPSel, MIPS64, and X86. Table IV presents the firmware information of the target devices.

Environment: FIRMPASS is evaluated on a PC with an Intel core i5-10400F 2.90 GHz CPU and 16GB RAM, running Ubuntu20.04.

B. Overall Results

Identified Vulnerabilities: In total, FIRMPASS identified 67 vulnerabilities from 615 firmware images, including 37 previously unknown vulnerabilities. Table III presents the details of vulnerabilities identified by FIRMPASS with CVE-ID. Among them, the number of vulnerabilities caused by Plaintext Storage of a Password, Weak Encoding for Password,

Use of Hard-coded Password, and Unverified Password Change are 9, 7, 8, and 6, respectively. As shown in Table V, 19 of the unknown vulnerabilities are due to Plaintext Storage of a Password, 11 are Weak Encoding for Password, 5 are Use of Hard-coded Password, and 2 are caused by Unverified Password Change. These identified vulnerabilities can lead to severe consequences such as privacy breaches and even the remote control of IoT devices.

Additionally, we found that these unknown vulnerabilities affect multiple different firmware versions of the same device model during evaluation. The details of the firmware version are shown in Table VI. For all the identified vulnerabilities, we have responsibly disclosed the details to the device vendors. We sent reports to 14 vendors, of which 6 have replied to us and 8 have not yet responded definitively. For security concerns, we have not made the details of the issues public. As soon as there is a definitive response, we will synchronize the update to FIRMPASS's github.²

Performance of FIRMPASS: Overall, FIRMPASS is fast and efficient in identifying broken password management. The shortest time spent for identification is only 11 s, while the longest is 47 s. We present the time taken for preprocessing, CPG generation, and querying required for 10 firmware from the tested dataset, as shown in Fig. 4. On average, the proportion of time spent on preprocessing, CPG generation, and query language is almost 60%, 16%, and 24%, respectively. The time is mostly spent on preprocessing because of the complexity of firmware images and the size of the target binaries, requiring more time to unpack and decompile.

²<https://github.com/huangjijing1106/FIRMPASS>

TABLE V
FIRMWARE MODELS WITH UNKNOWN VULNERABILITIES

Device ID	Vendors	Vulnerabilities Type
WNDR3400	Netgear	Plaintext Password Storage
DWR921	D-Link	Plaintext Password Storage
360P3	360	Plaintext Password Storage
360POP-P0	360	Plaintext Password Storage
360POP-P1	360	Plaintext Password Storage
WNDR4500	Netgear	Plaintext Password Storage
WN3000RP	Netgear	Plaintext Password Storage
WNR3500L	Netgear	Plaintext Password Storage
wnr854t	Netgear	Plaintext Password Storage
WN804	Netgear	Plaintext Password Storage
wgt624	Netgear	Plaintext Password Storage
dude-6.42.9-arm	mikrotik	Plaintext Password Storage
routers-arm-6.42.9	mikrotik	Plaintext Password Storage
routers-powerpc-6.42.9	mikrotik	Plaintext Password Storage
wsl1166dd	buffalo	Plaintext Password Storage
wsl600dd	buffalo	Plaintext Password Storage
DYRH08	linksys	Plaintext Password Storage
E2100L	linksys	Plaintext Password Storage
E1500	linksys	Plaintext Password Storage
US_AC6	Tenda	Weak Encoding for Password
RAX40	Netgear	Weak Encoding for Password
DIR640	D-Link	Weak Encoding for Password
DIR816	D-Link	Weak Encoding for Password
DIR878	D-Link	Weak Encoding for Password
TL-WR840	TP-Link	Weak Encoding for Password
TL-TR960	TP-Link	Weak Encoding for Password
WAC-124	Netgear	Weak Encoding for Password
BRT-AC828	ASUS	Weak Encoding for Password
A3002RU	TOTOLINK	Weak Encoding for Password
CS-W50HD	Planex	Weak Encoding for Password
DIR601	D-Link	Use of Hard-coded Password
TL-WR1043ND	TP-Link	Use of Hard-coded Password
TL-TR960	TP-Link	Use of Hard-coded Password
TL-TRXDR6020	TP-Link	Use of Hard-coded Password
whr300hp2d	buffalo	Use of Hard-coded Password
WNDR3400	Netgear	Unverified Password Change
WAC-124	Netgear	Unverified Password Change

TABLE VI
FIRMWARE MODELS WITH DIFFERENT VERSIONS

Models	Versions	Vulnerabilities
WNR3500	L-V1.2.2.40_34.0.48	Plaintext Password Storage
WNR3500	Lv2-V1.2.0.20_40.0.68	Plaintext Password Storage
dude	6.42.9	Plaintext Password Storage
dude	6.42.9-arm	Plaintext Password Storage
dude	6.42.9-tile	Plaintext Password Storage
dude	6.43.2	Plaintext Password Storage
dude	6.43.2-arm	Plaintext Password Storage
dude	6.43.2-tile	Plaintext Password Storage
dude	6.44beta17	Plaintext Password Storage
dude	6.44beta17-arm	Plaintext Password Storage
dude	6.44beta17-tile	Plaintext Password Storage
routers	arm-6.42.9	Plaintext Password Storage
routers	arm-6.43.2	Plaintext Password Storage
routers	arm-6.44beta17	Plaintext Password Storage
routers	mipsbe-5.26	Plaintext Password Storage
routers	mipsbe-6.42.9	Plaintext Password Storage
routers	powerpc-6.44beta17	Plaintext Password Storage
routers	mipsbe-6.43.2	Plaintext Password Storage
routers	powerpc-6.43.2	Plaintext Password Storage
routers	mipsbe-6.44beta17	Plaintext Password Storage
routers	powerpc-6.42.9	Plaintext Password Storage
routers	powerpc-5.26	Plaintext Password Storage
360POP	P0-V2.0.50.50364	Plaintext Password Storage
360POP	P1-V2.0.55.49421	Plaintext Password Storage
360POP	P2-V2.0.50.48532	Plaintext Password Storage
360POP	R1-V2.0.16.50311	Plaintext Password Storage
DIR-816L	REVA_PATCH_1.01.B01	Weak Encoding for Password
DIR-816L	REVB_2.00B01	Weak Encoding for Password
DIR-816L	REVB_2.01B03_WW	Weak Encoding for Password
DIR-816L	REVB_2.03B03	Weak Encoding for Password
DIR-816L	REVB_2.05.B02	Weak Encoding for Password
DIR-816L	REVB_PATCH_2.06.B01	Weak Encoding for Password
DIR-601	REVA_1.01	Use of Hard-coded Password
DIR-601	REVA_1.02	Use of Hard-coded Password
DIR-601	REVA_1.04B02	Use of Hard-coded Password
DIR-601	REVB_2.01	Use of Hard-coded Password
DIR-601	REVB_v2.02B01	Use of Hard-coded Password

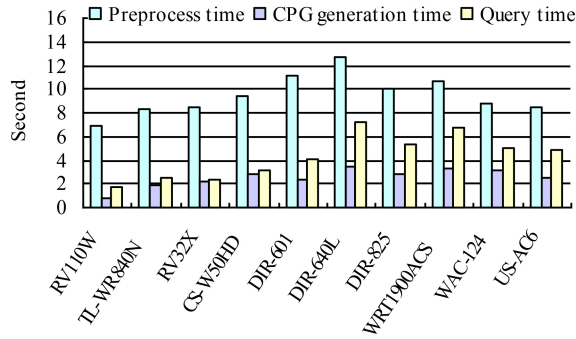


Fig. 4. Execution time of FIRMPASS.

Specifically, the preprocessing of FIRMPASS took an average of 8.4 s. with firmware unpacking and Ghidra analysis accounting for 38% and 61% of the preprocessing time, respectively. FIRMPASS requires 5.6 s to execute a query for each program recovered by Ghidra. The query execution time overhead includes: 1) the time required to import the decompiled JSON file output by Ghidra to generate the CPG, averaging 2–3 s and producing approximately 104 720 CPG nodes on average and 2) the time required to search for paths, ranging from less than 1–9 s.

C. Accuracy

FIRMPASS reported a total of 82 cases in 615 firmware images, and we confirmed 67 of them are vulnerable. We

manually check the 82 cases to identify false positives. For false negatives, we first search for binaries in the firmware database that may be relevant to authentication using keywords, and then sample them to identify false negatives.

False Positives: The introduction of Ghidra [21] enhances the cross-architectural analysis and scalability of the FIRMPASS, but it also leads to some false positives. For example, FIRMPASS identified the hash function `crypt()` used for password management in program *Busybox* of the device model TL-WR1043ND in Listing 2. This hash function is provided by the library *libcrypt-0.9.30.so* and selects the hash algorithm based on the salt pattern used, such as MD5, DES, SHA-256, and SHA-512. However, FIRMPASS cannot identify the hash algorithm used based on the specific function callsites and implementations of the function `crypt()`. This causes the `crypt()` function, even when using secure hash algorithms like SHA-256 and SHA-512, to be incorrectly reported by the tool as weak encoding for passwords.

False Negatives: The false negatives occur due to the following reasons.

- 1) **Decompilation Errors:** FIRMPASS defines a set of queries based on the established password management model to match the decompiled results recovered by Ghidra in the C-Code level. However, data flow interruptions caused by decompilation errors can lead to false negatives for the queries. For example, assigning a variable to a void-type function or not returning a value

```

1 void pw_encrypt(char *param_1, char *param_2)
2 {
3     char *pcVar1;
4     free(DAT_004b8d30);
5     pcVar1 = crypt(param_1, param_2);
6     DAT_004b8d30 = (void *)xstrdup(pcVar1);
7     return;
8 }
9
10 char * crypt(char *__key, char *__salt)
11 {
12     char *pcVar1;
13     code *UNRECOVERED_JUMPTABLE;
14     if ((*__salt == '$') && (__salt[1] == '1')) && {
15         __salt[2] == '$') {UNRECOVERED_JUMPTABLE =
16         FUN_00011c80;}
17     else { UNRECOVERED_JUMPTABLE = FUN_00011548; }
18     pcVar1 = (char *) (*UNRECOVERED_JUMPTABLE) (__key,
19     __salt);
20     return pcVar1;
21 }

```

Listing 2. Code snippet from device model TL-WR1043ND.

from a nonvoid-type function can interrupt the data flow, leading to a mismatch in the queries. When we locate some system functions by the queries, such as locating the password input or storage (`read()`/`write()`) and storage file (*password* and *shadow*), there will be query failure due to the error of decompilation.

- 2) *Nonstandard Function Implementations*: Some vendors utilize self-customized components with nonstandard function implementations for password management. It is impractical to define all self-customized components, leading to potential issues in these cases that FIRMPASS cannot detect. However, these components are not common, which means the real-world impact of these false negatives is deemed acceptable.

D. Case Study

FIRMPASS identified *CVE-2021-20171* in device model Netgear WAC124 (AC2000), a plaintext password storage vulnerability that can disclose device administrator passwords. As shown in Listing 3, the function `FUN_00455ed8()` in program *Busybox* is used for user authentication. The program checks the validity of the username in lines 8–14. Variable `__s2` stores the information related to the user's password. The function `FUN_00454ab8()` is to obtain the user input. The program verifies the plaintext password input by the user with the stored password in line 19, without any hash operations. It means that the password is stored in plaintext, allowing attackers to retrieve it from configuration files or password databases.

FIRMPASS exposes this vulnerability by identifying the data flow of the authentication process below. FIRMPASS first determines if `__s1` and `__s2` in the `strcmp()` are related to the password. Then, we identify whether the two parameters are fetched from the password configuration file or from the `getspname()` through the data flow. Once it has been identified using the query that `__s2` is the stored password for the read, then output the data flow of `__s1` from `read()` to `strcmp()`, and query this data flow for the presence of hash function. As shown in Listing 3, there is no hash function from `FUN_00454ab8()` to `strcmp()`, indicating that there is a

```

1 bool FUN_00455ed8(int param_1)
2 {
3     bool bVar1;
4     char *__s1;
5     int iVar2;
6     char *__s2;
7
8     if (param_1 == 0) {
9         __s2 = "aa";
10    }
11    else {
12        __s2 = *(char **) (param_1 + 4);
13        if (*__s2 == '\0') {
14            return true;
15        }
16    }
17    __s1 = (char *)FUN_00454ab8(0, "Password: ");
18    bVar1 = false;
19    if (__s1 != (char *)0x0) {
20        iVar2 = strcmp(__s1, __s2);
21        bVar1 = iVar2 == 0;
22    }
23    return bVar1;
24 }

```

Listing 3. Firmware WAC124(AC2000) decompiled code in CVE-2021-20171.

comparison of user plaintext passwords, leading to a Plaintext Storage of a Password vulnerability.

VII. RELATED WORKS

This section reviews prior research on the detection of access control issues in firmware and the security of password management.

A. Detection of Access Control Issues in Firmware

Researchers have been intensely interested in IoT firmware analysis [22], [23], [24], [25], [26] in recent years. For access control issues in interactions between device and cloud, Chen et al. [27], [28] proposed two models to decompose the remote binding process, revealed various design flaws in existing remote binding device schemes, and assessed their security risks. FirmRes [29] was an automatic firmware analysis tool for identifying broken device-cloud access control in firmware by reconstructing the device-cloud messages. Zhang et al. [13] captured custom encryption functions and encryption misuse by identifying low-level encryption API parameters using call rules and stack layouts. Egele et al. [12] used static program slicing to identify the connection between the initial information of password encryption and encryption operations for the detection of typical password misuse. Firmalice [7] was a firmware analysis framework that automatically performs symbolic execution on the sliced program to detect potential backdoors and privilege-escalation vulnerabilities in embedded devices. IoTScope [30] constructed probing requests through firmware analysis to test physical devices and uses differential analysis to filter out irrelevant requests and interfaces, thereby revealing hidden Web interfaces in IoT devices. Redini et al. [31] detected unsafe interactions and identified vulnerabilities by modeling and tracking multibinary interactions, propagating tainted information between binary files. It can contribute to the discovery of access control problems at the binary file interaction level. *KingFisher* [32] identified and checked whether shared credentials violate the security properties that should be adhered to, exposing the security issues of shared credentials in IoT mobile communications.

Our research focuses on vulnerabilities in password management within the program's code logic. To the best of our knowledge, we are the first to define rules and a general model to standardize the entire password management process, and utilize formal methods represented through queries to expose vulnerabilities.

B. Security Analysis of Password Management

For the research of password management, other researchers have also done a lot of research work in this field. Zhang et al. [11] analyzed how applications use permissions to access permission-sensitive resources and how these resources are further utilized by the application. With analysis of permission use behaviors, it is efficient for security analysts to assess the internally sensitive behaviors of an app. Raponi and Di Pietro [33] investigated potential attacks on online password recovery systems by analyzing the mechanisms employed by several most popular websites. Sakashita et al. [34] analyzed the risks of password management quantitatively through a mathematical model. Hatzivasilis [35] presented a comprehensive overview of password management theory and provided a practical guide for mobile and Web application developers, as well as IT organizations, on implementing secure login processes. To prevent weak password attacks, He et al. [36] revealed that the current password software platforms overestimate the strength of passwords with special characters. They made effective recommendations for users and IoT platforms to enhance password security. Sun et al. [37] used neural cryptosystems to enhance the privacy security of authentication, encryption and key distribution methods for IoT devices. Zhang et al. [38] proposed a dual-server architecture and a three-lock security policy to ensure secure reliable key storage and key transmission based on password authentication for the IoT devices in the 5G networks. Lian et al. [39] proposed a new security model to enhance the security of the password authenticated key exchange protocol and ensure that the transmitted records and password files do not reveal identity information.

The context of FIRMPASS is within IoT firmware, whose logical flow of password management is different from the above Web applications and mobile applications. As a result, the methodology is also different.

VIII. CONCLUSION

In this article, We have presented FIRMPASS, an automated tool for detecting password management vulnerabilities in IoT devices. We have defined the password management security rules and established a password management model. This helps us to systematically define queries to reason about the code properties and identify vulnerabilities. Evaluated with 615 real-world firmware images, FIRMPASS successfully identified 67 vulnerabilities, including 37 previously unknown ones.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful comments.

REFERENCES

- [1] (Data Prot. Comm., Dublin, Ireland). *Irish Data Protection Commission Fines Meta Ireland 91 Million*. (2024). [Online]. Available: <https://www.dataprotection.ie/en/news-media/press-releases/DPC-announces-91-million-fine-of-Meta>
- [2] "Evernote password reset notification." Evernote. 2013. [Online]. Available: <https://www.ktcdigital.com/evernote-password-reset-notification/>
- [3] (LinkedIn Corp., Sunnyvale, CA, USA). *Data Stolen from LinkedIn in 2012*. May 17, 2016. [Online]. Available: <https://www.linkedin.com/help/linkedin/answer/a1338522?hcppcid=search>
- [4] T. SpiderLabs, (Trustwave, Chicago, IL, USA). *eHarmony Password Dump Analysis*. Jun. 26, 2012. [Online]. Available: <https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/eharmony-password-dump-analysis/>
- [5] (Twitter, San Francisco, CA, USA). *Update About an Alleged Incident Regarding Twitter User Data Being Sold Online*. Jun. 26, 2012. [Online]. Available: <https://privacy.x.com/en/blog/2023/update-about-an-alleged-incident-regarding-twitter-user-data-being-sold-online>
- [6] "AntiSec." [Online]. Available: https://en.wikipedia.org/wiki/Operation_AntiSec
- [7] Y. Shoshitaishvili, R. Wang, C. Hauser, C. Kruegel, and G. Vigna, "Firmalace-automatic detection of authentication bypass vulnerabilities in binary firmware," in *Proc. NDSS*, 2015, p. 1.
- [8] E. Dauterman, H. Corrigan-Gibbs, D. Mazières, D. Boneh, and D. Rizzo, "True2F: Backdoor-resistant authentication tokens," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 398–416.
- [9] C. Qin et al., "UCRF: Static analyzing firmware to generate under-constrained seed for fuzzing SOHO router," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103157.
- [10] F. Schuster and T. Holz, "Towards reducing the attack surface of software backdoors," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 851–862.
- [11] Y. Zhang et al., "Vetting undesirable behaviors in android apps with permission use analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 611–622.
- [12] M. Egele, D. Brumley, Y. Fratantonio, and C. Kruegel, "An empirical study of cryptographic misuse in android applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 73–84.
- [13] L. Zhang, J. Chen, W. Diao, S. Guo, J. Weng, and K. Zhang, "CryptoREX: Large-scale analysis of cryptographic misuse in IoT devices," in *Proc. 22nd RAID*, 2019, pp. 151–164.
- [14] "Linux-PAM." [Online]. Available: <https://Linux-pam.org/Linux-PAM-html/sag-introduction.html>
- [15] "Joern." [Online]. Available: <https://joern.io/>
- [16] T. Rahat, Y. Feng, and Y. Tian, "Cerberus: Query-driven scalable security checking for OAuth service provider implementations," in *Proc. 29th ACM Conf. Comput. Commun. Secur. (CCS)*, 2022, pp.2459–2473.
- [17] "Horn clause." [Online]. Available: https://simple.wikipedia.org/wiki/Horn_clause
- [18] "Backus Naur." [Online]. Available: https://en.wikipedia.org/wiki/BackusNaur_form
- [19] J. W. Lloyd, *Foundations of Logic Programming*. Berlin, Germany: Springer, 2012.
- [20] "Binwalk." [Online]. Available: <https://github.com/ReFirmLabs/binwalk>
- [21] "Github—Nationalecurityagency/ghidra: Ghidra is a software reverse engineering (SRE) framework." Ghidra, Accessed: Oct. 17, 2022. [Online]. Available: <https://github.com/NationalSecurityAgency/ghidra>
- [22] O. Alrawi, C. Lever, M. Antonakakis, and F. Monrose, "Sok: Security evaluation of home-based IoT deployments," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 1362–1380.
- [23] F. Chen, D. Luo, T. Xiang, P. Chen, J. Fan, and H.-L. Truong, "IoT cloud security review: A case study approach using emerging consumer-oriented applications," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–36, 2022. [Online]. Available: <https://doi.org/10.1145/3447625>
- [24] K. L. Wu, M. H. Hue, K. F. Tang, and S. Y. Chau, "The devil is in the details: Hidden problems of client-side enterprise Wi-Fi configurators," in *Proc. 16th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2023, pp. 251–261.
- [25] Q. Wang et al., "MPInspector: A systematic and automatic approach for evaluating the security of IoT messaging protocols," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 4205–4222.
- [26] D. Bonaventura, S. Esposito, and G. Bella, "Smart bulbs can be hacked to hack into your household," 2023, *arXiv:2308.09019*.

- [27] J. Chen, M. Sun, and K. Zhang, "Security analysis of device binding for IP-based IoT devices," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops (PerCom Workshops)*, 2019, pp. 900–905.
- [28] J. Chen et al., "Your IoTs are (not) mine: On the remote binding between IoT devices and users," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2019, pp. 222–233.
- [29] Y. Xiao, J. Chen, Y. Hu, and J. Huang, "FIRMRES: Exposing broken device-cloud access control in IoT through static firmware analysis," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2024, pp. 900–905.
- [30] W. Xie et al., "Game of hide-and-seek: Exposing hidden interfaces in embedded Web applications of IoT devices," in *Proc. ACM Web Conf.*, 2022, pp. 524–532.
- [31] N. Redini et al., "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *Proc. IEEE Symp. Security Privacy (SP)*, 2020, pp. 1544–1561.
- [32] Y. Zhang, S. Ma, J. Li, D. Gu, and E. Bertino, "KingFisher: Unveiling insecurely used credentials in IoT-to-mobile communications," in *Proc. 52nd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2022, pp. 488–500.
- [33] S. Raponi and R. Di Pietro, "A longitudinal study on Web-sites password management (in)security: Evidence and remedies," *IEEE Access*, vol. 8, pp. 52075–52090, 2020.
- [34] K. Sakashita, A. Kawagoe, and T. Okuda, "A risk evaluation of password management strategies," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, 2020, pp. 1–2.
- [35] G. Hatzivasilis, "Password management: How secure is your login process?" in *Proc. 2nd Int. Workshop, Model-Driven Simul. Train. Environ. Cybersecur. (MSTEC)*, 2020, pp. 157–177.
- [36] D. He, Z. Liu, S. Zhu, S. Chan, and M. Guizani, "Special characters usage and its effect on password security," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19440–19453, Jun. 2024.
- [37] Y. Sun, F. P.-W. Lo, and B. Lo, "Lightweight Internet of Things device authentication, encryption, and key distribution using end-to-end neural cryptosystems," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 14978–14987, Aug. 2022.
- [38] S. Zhang, Y. Liu, T. Gao, Y. Xie, and C. Zhou, "Practical and secure password authentication and key-agreement-scheme-based dual server for IoT devices in 5G network," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34639–34651, Nov. 2024.
- [39] H. Lian, Y. Yang, and Y. Zhao, "Efficient and strong symmetric password authenticated key exchange with identity privacy for IoT," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 4725–4734, Mar. 2023.



Jiongyi Chen received the B.Eng. and Ph.D. degrees from Chinese University of Hong Kong, Hong Kong, in 2015 and 2019, respectively.

He is currently a Lecturer with the National University of Defense Technology, Changsha, China. His research focuses on software security.



Yuting Xiao received the master's degree from the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China.

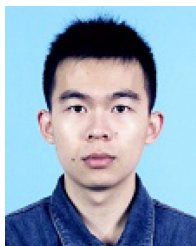
Her research interest is IoT embedded device security.



Zheng Qin (Member, IEEE) received the Ph.D. degree in computer software and theory from Chongqing University, Chongqing, China, in 2001.

From 2010 to 2011, he served as a Visiting Scholar with the Department of Computer Science, Michigan University, Ann Arbor, MI, USA. He is a Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China, where he serves as a Vice Dean. He also serves as the Director with the Hunan Key Laboratory of Big Data Research and Application.

His main interests are network and data security, privacy, data analytics and applications, and applied cryptography.



Jing Huang is currently pursuing the Ph.D. degree from the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China.

His research interest is IoT embedded device security.



Yupeng Hu (Senior Member, IEEE) is currently a Professor and a Ph.D. Supervisor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His main research interest focuses on software and hardware integrated system security and reliability, including AI system security, storage system security, and PUF/CPU security and reliability.